

Proceso de Diseño

Lo primero que tuvimos que realizar en este laboratorio, fue la implementación del procesador monociclo teniendo en cuenta sus diferentes componentes o recursos hardware utilizados en todo el proceso de ejecución de una instrucción de la arquitectura MIPS en cada ciclo de reloj, incluidos y resumidos en la datapath (ruta de datos) que sigue el camino indicado por la unidad de control que es el otro componente incluido dentro del procesador y que actúa como el cerebro del mismo, porque es el que decide cómo implementar la ejecución de la instrucción dentro del camino de datos según el tipo de formato de la instrucción MIPS.

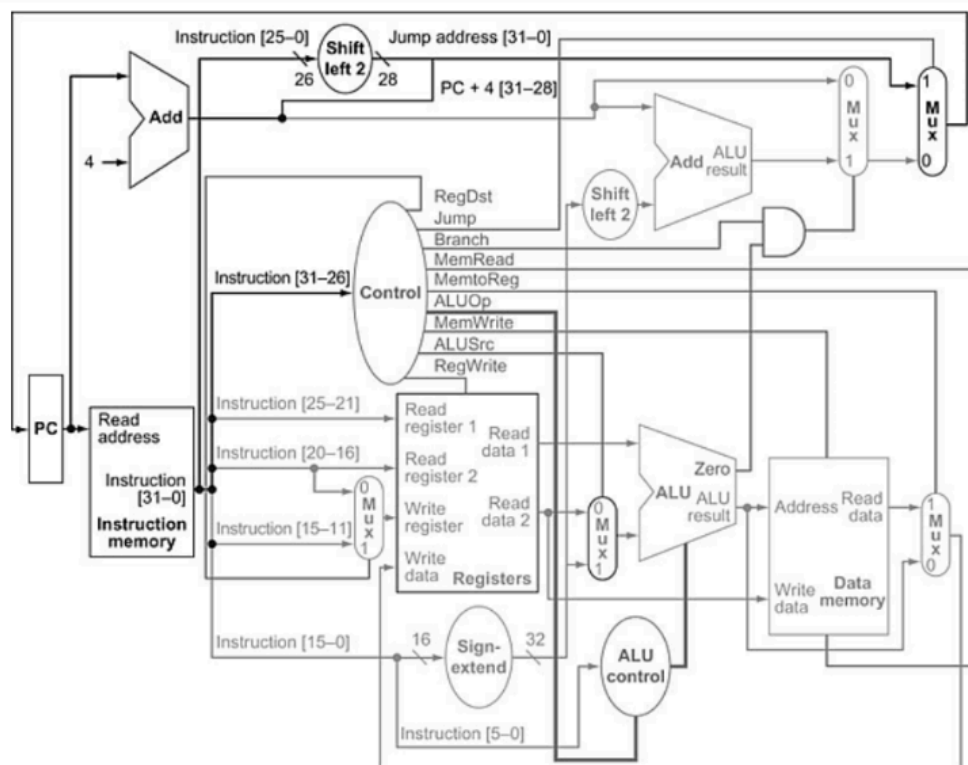
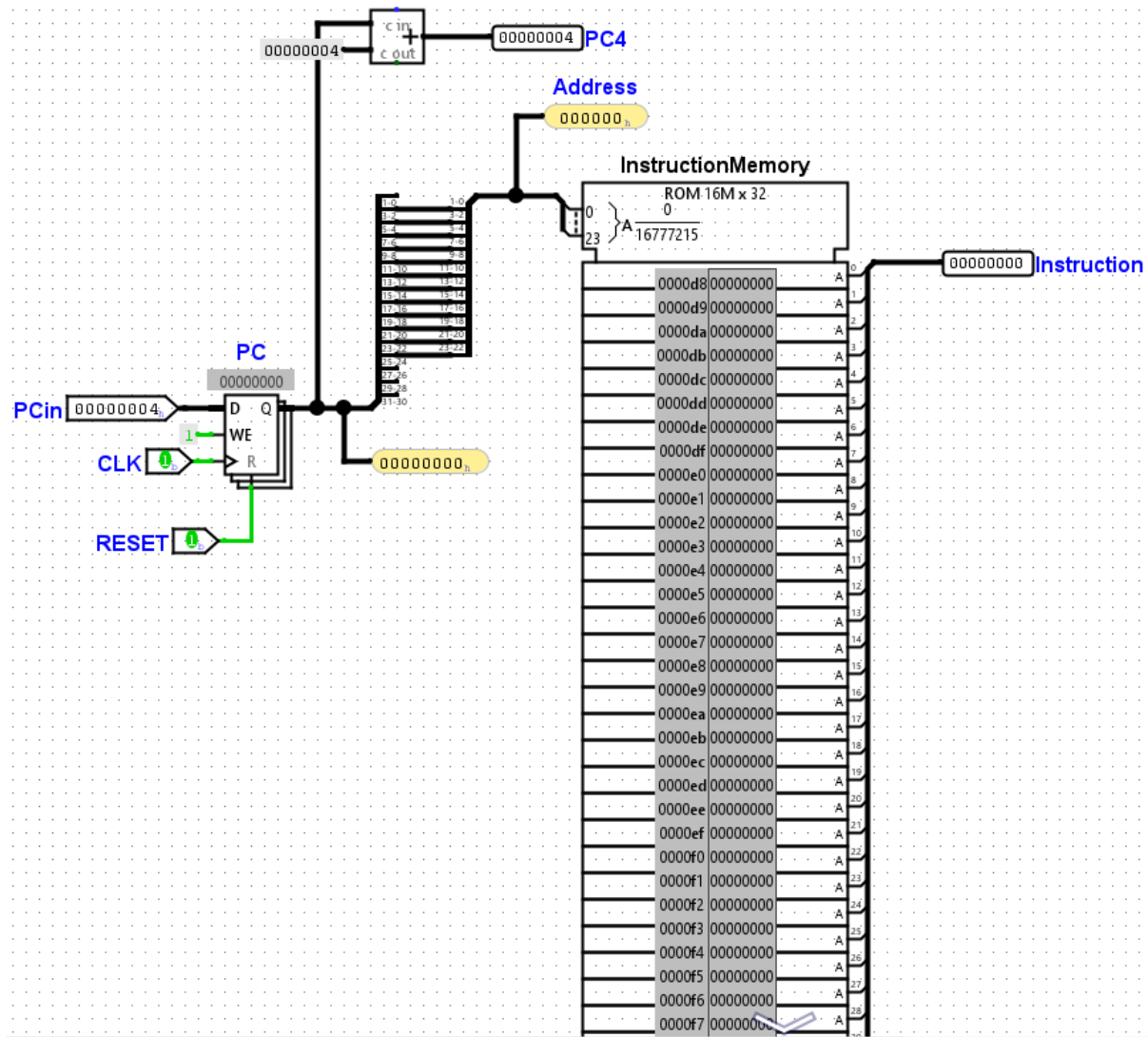


Figura 1. Implementación monociclo de la arquitectura MIPS capaz de ejecutar instrucciones lw, sw, add, sub, and, or, nor, slt, beq y jump.

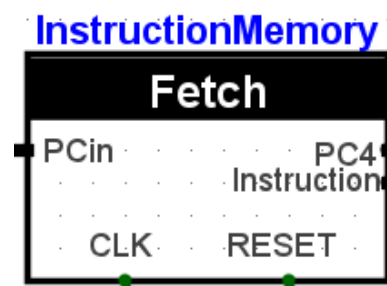
En este camino de implementación, seguimos los principios de los campos en que se divide una instrucción MIPS32, según los tres tipos de formatos de una instrucción, sea tipo R, tipo I o tipo J, donde según el tipo de formato de la instrucción indicada cada una cuenta con sus respectivos campos, con sus códigos de operación, de identificador de registros y demás valores de los campos definidos en el documento *MIPS Reference Data*.

Al momento de implementar el procesador monociclo, lo hicimos siguiendo las diferentes etapas que utilizan diferentes recursos hardware a lo largo que una instrucción MIPS atraviesa todo el camino de datos seguido por el procesador.

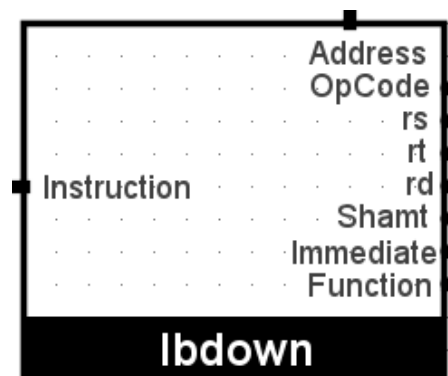
Como componente o recurso hardware inicial tenemos la memoria de instrucciones y la etapa de fetch (búsqueda de instrucciones), que es donde se busca donde inicia el recorrido por el datapath con una instrucción MIPS, en esta etapa mediante un Counter Program (Contador de Programa) se indicará qué instrucción se trae de dicha memoria, entregándonos un número de 32 bits con la instrucción por ejecutar, posteriormente, el valor de PC se actualizará mediante un sumador con $PC+4$ luego de haber tomado la instrucción, para continuar el modo de ejecución secuencial, esto en el caso de una instrucción tipo R.

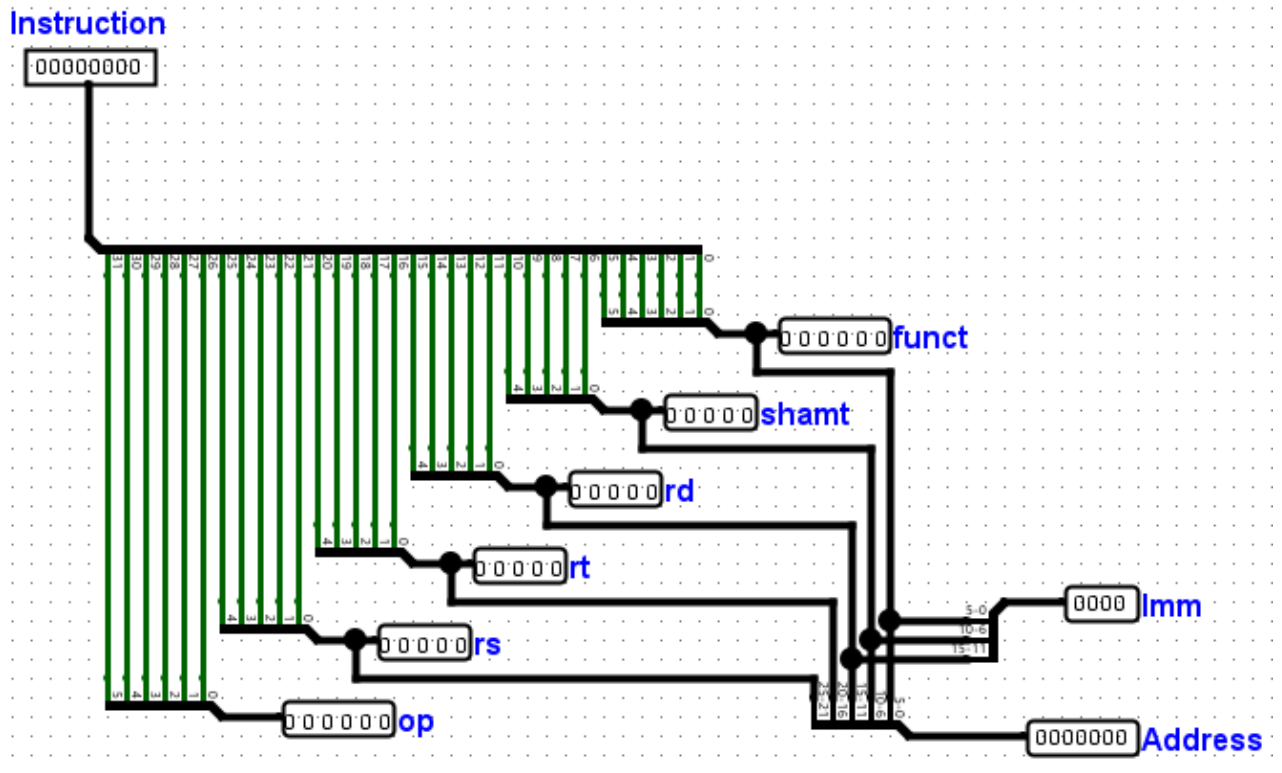


La búsqueda de instrucciones implica leer una instrucción de la memoria de instrucciones utilizando la dirección indicada por el registro PC. El Counter Program almacena la dirección de la instrucción que se está ejecutando. Además, también podremos calcular con este valor de PC+4 las direcciones de destino de salto. En esta etapa el componente hardware utilizado es una memoria ROM debido a que en esta etapa solo podremos leer datos de la memoria, además tendremos 24 bits de direccionamiento que es lo máximo que nos ofrece Logisim, por lo que tendremos que hacer un split de la instrucción de 32 bits que nos entre de la memoria de instrucciones y adecuarla a 24 bits de entrada de direccionamiento en la ROM.



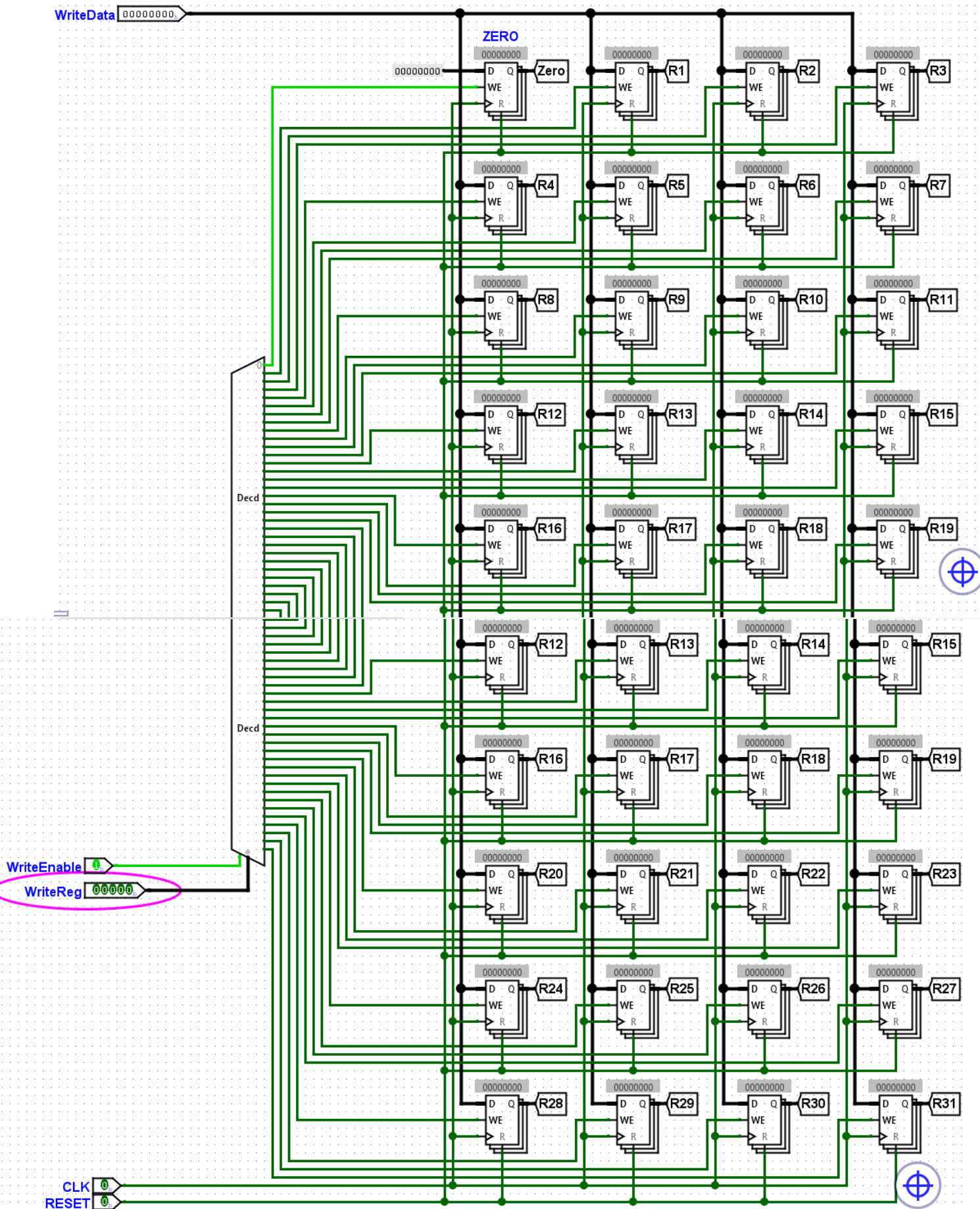
Una vez tenemos la instrucción de la memoria de instrucciones que es un número de 32 bits que corresponde a la instrucción que va a ser ejecutada, tenemos que descomponer esta instrucción de 32 bits según los campos de los diferentes formatos de instrucción, para así posteriormente poder pasarla a la Unidad de Control y al Banco de Registros en donde se tomarán los valores de los registros dependiendo del tipo de instrucción. El componente lbdwn es quien identifica el tipo de instrucción y la particiona según sus diferentes campos.

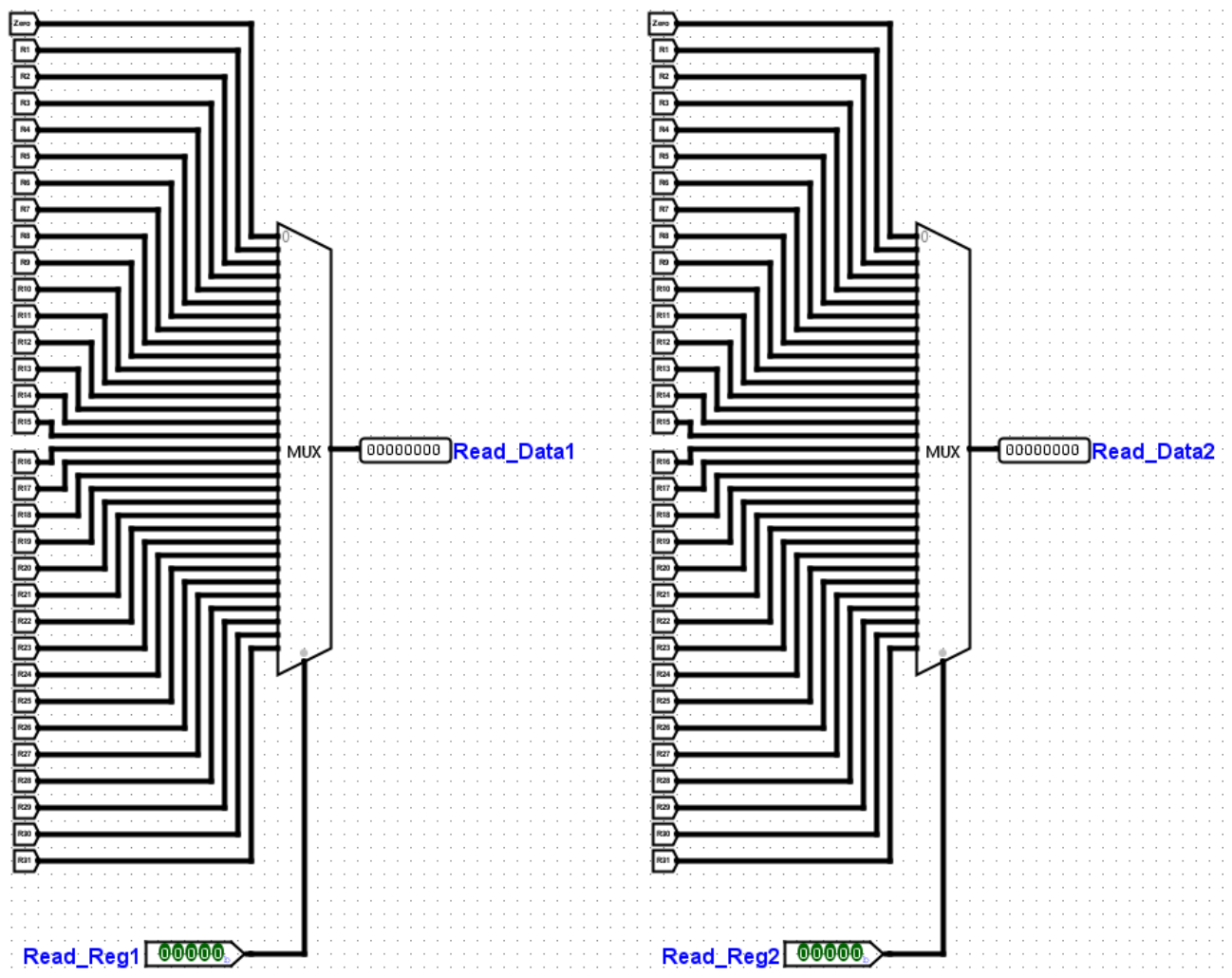




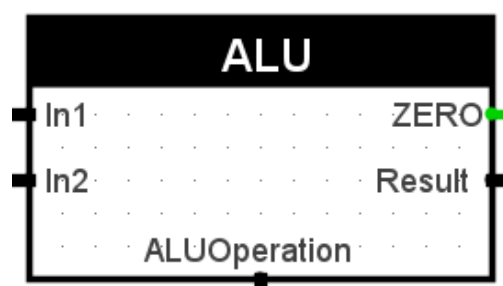
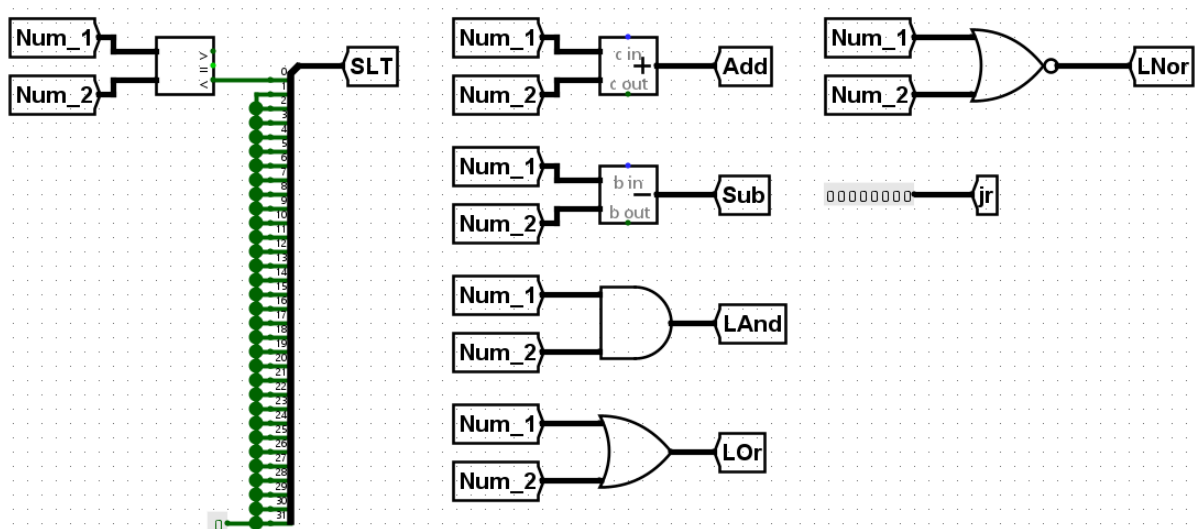
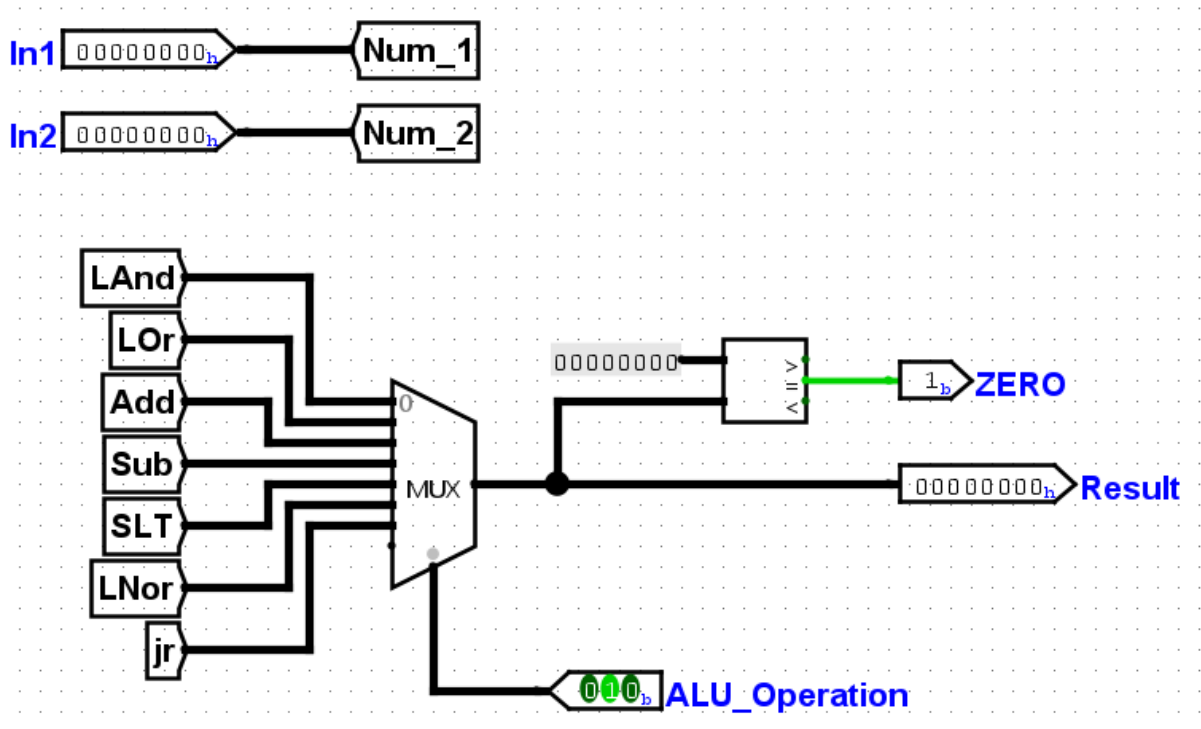
En esta etapa necesitamos del componente del Banco de Registros que es donde hacemos lectura de los operandos según sea el caso. Este banco agrupa los 32 registros definidos por la arquitectura MIPS y nos permitirá el acceso simultáneo a 3 registros, dos de acceso a lectura con entradas de 5 bits cada una y uno de acceso a escritura a un registro con 5 bits de igual manera. El banco de registros contará con una con una señal de control de escritura explícita, la cual nos indicará si utilizamos los registros para cargar un valor o almacenarlo.



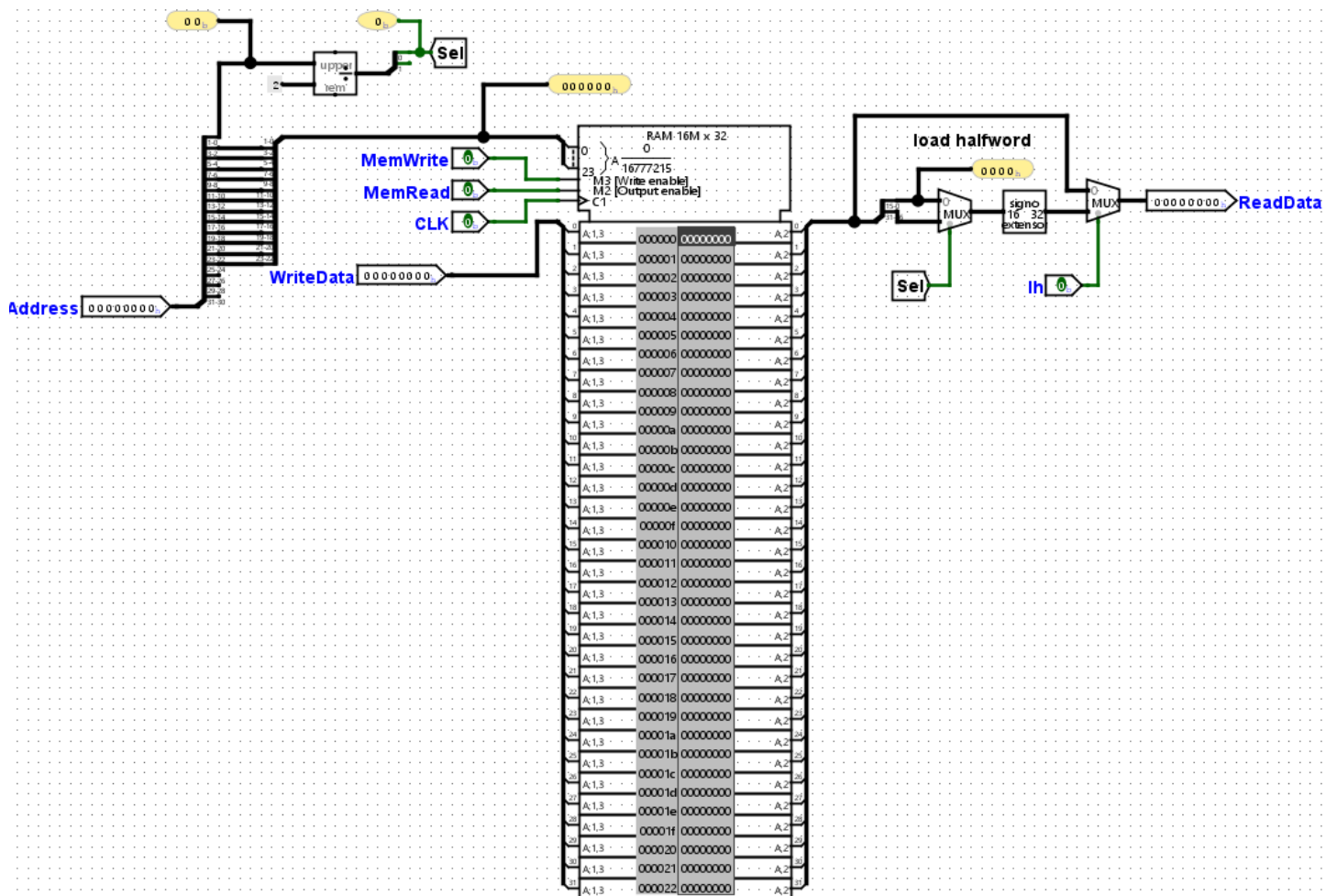




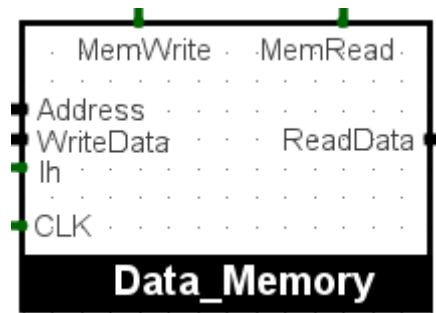
Una vez la máquina interpreta los operandos son necesariamente llevados hacia la ALU para realizar los debidos cálculos y operaciones, la cual nos indicará el tipo de operación sobre sus operandos de entrada mediante la señal de control de ALU operation. Este componente es utilizado de igual manera, tanto como para calcular las operaciones aritmético-lógicas, como la dirección efectiva de acceso a memoria y para hacer comparaciones de igualdad y calcular la dirección de la instrucción de salto condicional. Con la ALU siempre tendremos operandos de 32 bits de entrada y de resultado. La ALU contará con una señal de salida Zero de 1 bit que nos será muy útil y nos indicará si el resultado producido por la operación en la ALU es cero.



Después en el flujo de datos del CPU, una vez se ha pasado por la etapa de cálculos que utiliza el recurso de hardware de la alu, viene el componente de la memoria de datos a la que tienen acceso exclusivamente las instrucciones de acceso a memoria, en donde podemos leer y escribir en una dirección calculada por la ALU. La ejecución de las instrucciones load/store demanda memorias de datos con direcciones y datos de 32 bits, por lo cual se hace necesario un componente de extensión de signo para transformar un valor de 16 bits inducido por el campo offset de una instrucción de tipo inmediato en otro valor equivalente a 32 bits. De igual manera, nuestra memoria RAM, contará con dos señales de control las cuales nos indicarán si ejecutaremos instrucciones de lectura o de escritura en la memoria.

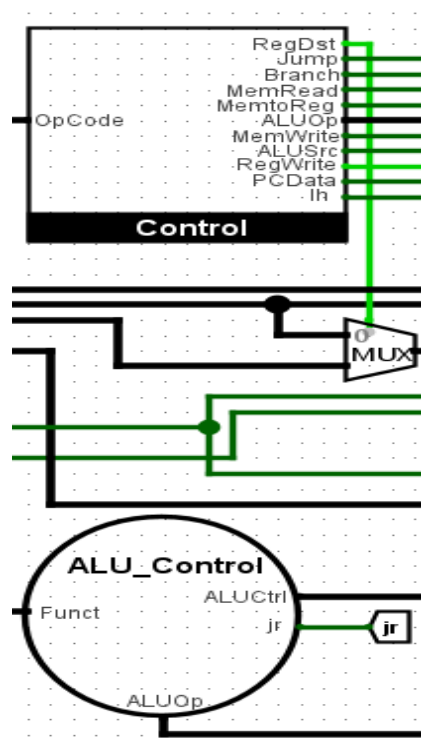


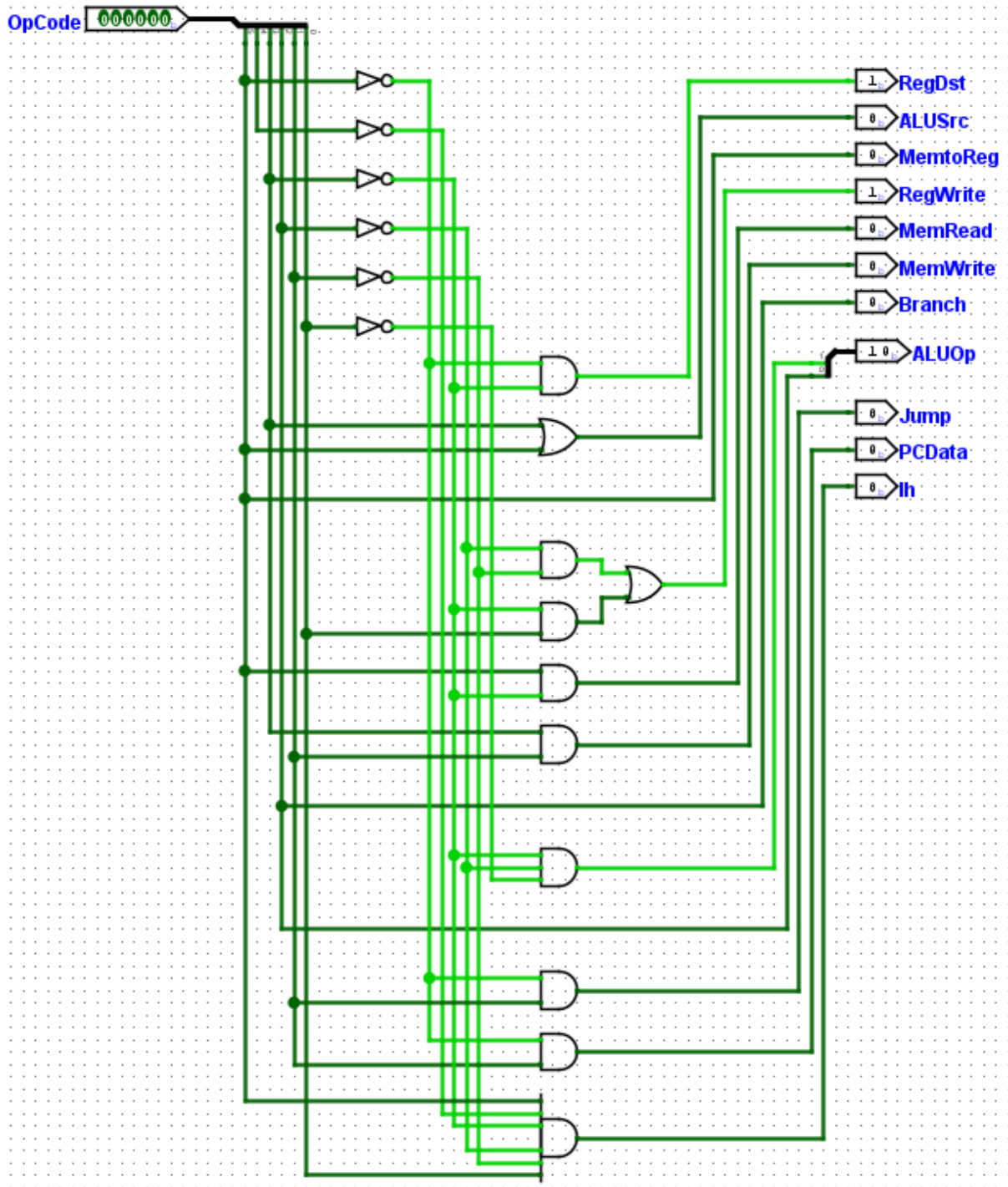
En el caso particular de nuestro equipo, nos tocó adicionar e implementar las instrucciones load halfword y add immediate.

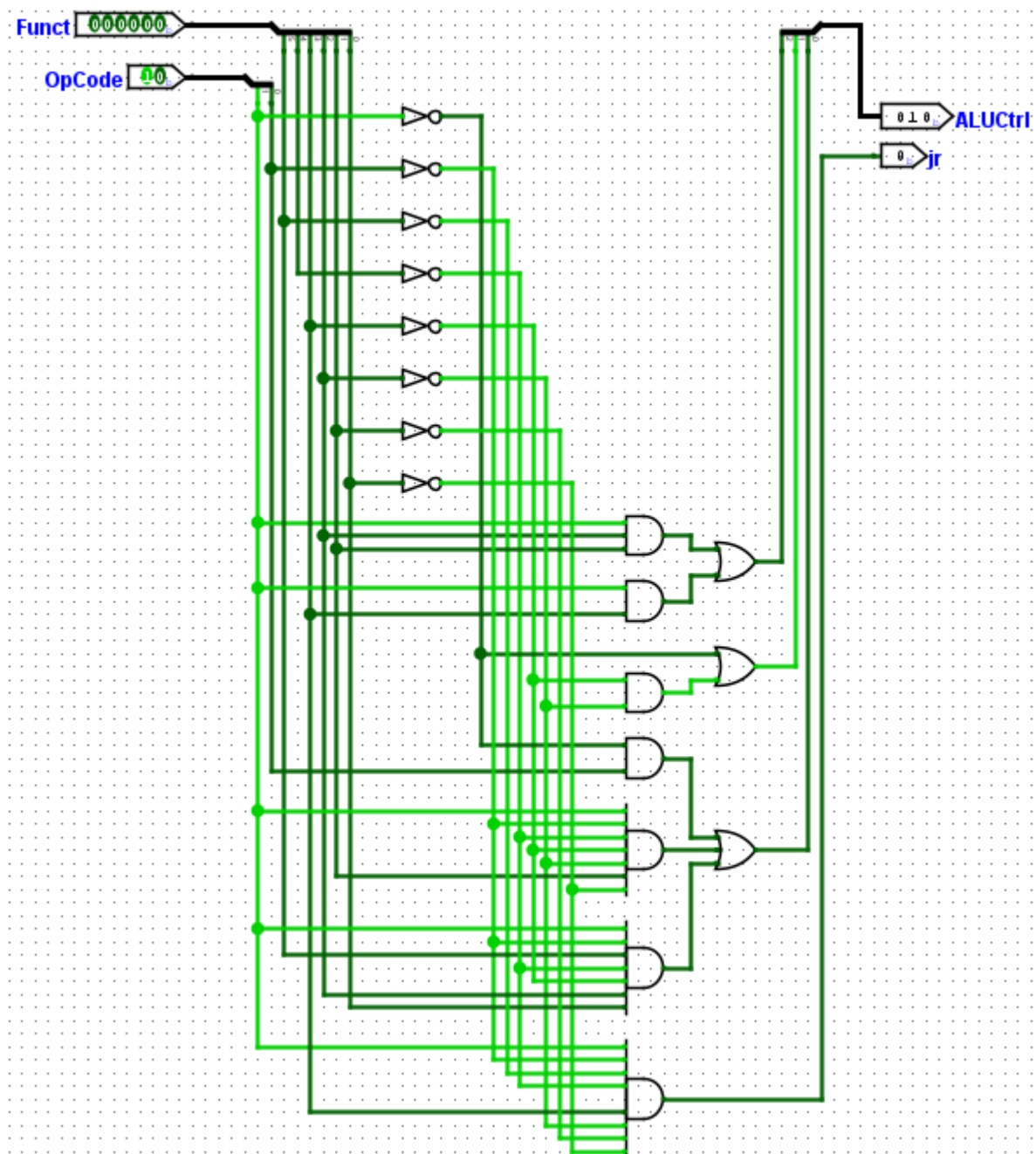


Posteriormente, si se lee el contenido de la memoria es dirijo el valor hacia el banco de registros para guardar el resultado de la ejecución de la instrucción y si es una instrucción store escribimos en una posición determinada de la memoria.

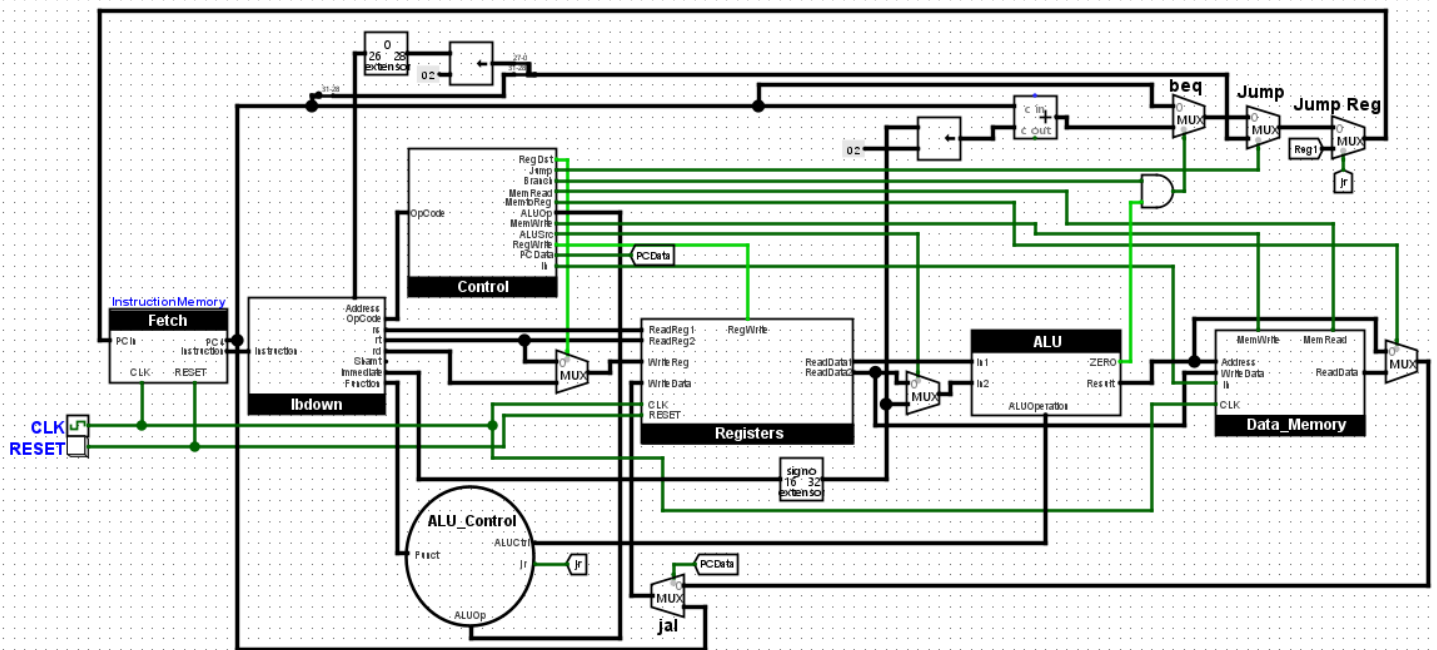
El cerebro del CPU queda determinado por la unidad de control que es la que nos indica cómo proceder durante la ejecución de una instrucción en todo el camino de datos. A esta unidad de control le añadimos un **ALU_Control** la cual determinará qué operación y cómo se procederá a ejecutar los cálculos en la ALU. Es necesario añadir unos componentes de multiplexores cuando un dato vaya camino a una unidad funcional de la cual puedan provenir de distintas fuentes, y para elegir la fuente de datos se emplea unas señales de control.







El procesador monociclo ejecuta por completo todas las instrucciones en un ciclo de reloj, lo que nos indica que la ejecución de una instrucción comienza en un flanco de subida del reloj y se da por terminada en el siguiente flanco de subida, de esta manera podremos hacer una actualización de los diferentes estados que atraviesan nuestra CPU.



Ejecución del programa en Mars

El programa que a nuestro grupo nos tocó implementar es el de tomar una matriz 5x5 de números enteros con valores entre -100 y 100, y determinar cuales son el mayor y el menor número almacenado en cada fila y en cada columna de la matriz. Para esto se realizó un código de prueba en MIPS32 el cual probaba el funcionamiento. Posteriormente se hizo una “traducción” del programa utilizando las direcciones y funciones correspondientes para que fuera compatible con nuestra versión del procesador de Logisim. El programa carga los siguientes datos:

```
.data

matriz: .word -18, 76, 39, 51, 13, -75, -81, 8, 96, 14, 99, -57, -49, 7, 5, 40, -28,
length: .word 5                                # Longitud filas y columnas

max_nums: .space 40
min_nums: .space 40
```

Carga la matriz, la dimensión de la matriz y guarda unos espacios para almacenar todos los valores mayores y menores encontrados. Luego estos datos quedan guardados en la memoria de Mars así:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	-18	76	39	51	13	-75	-81	8
0x10010020	96	14	99	-57	-49	7	5	40
0x10010040	-28	64	-99	-3	88	95	31	-87
0x10010060	-65	5	0	0	0	0	0	0
0x10010080	0	0	0	0	0	0	0	0
0x100100a0	0	0	0	0	0	0	0	0
0x100100c0	0	0	0	0	0	0	0	0
0x100100e0	0	0	0	0	0	0	0	0

Al terminar la ejecución, la memoria de datos queda de la siguiente manera:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	-18	76	39	51	13	-75	-81	8
0x10010020	96	14	99	-57	-49	7	5	40
0x10010040	-28	64	-99	-3	88	95	31	-87
0x10010060	-65	5	76	96	99	64	95	99
0x10010080	95	64	96	14	-18	-81	-57	-99
0x100100a0	-87	-75	-81	-49	-99	-65	0	0
0x100100c0	0	0	0	0	0	0	0	0
0x100100e0	0	0	0	0	0	0	0	0

Los primeros 10 datos después del número 5 (correspondiente a la dimensión de la matriz) son los 10 valores más grandes de la matriz, siendo los primeros 5 los mayores por filas y los segundos 5 los mayores por columna. Los otros 10 datos serían los menores valores de la matriz, los cuales siguen el mismo orden anterior.

Ejecución en Logisim

Para la ejecución del programa en Logisim, lo único que debe hacerse es cargar el archivo ROM.txt en la memoria de instrucciones y el archivo RAM.txt en la memoria de datos. Luego de esto, se puede simular el programa activando el reloj. Luego de que se ejecuten todas las instrucciones, podrá verse en la dataMemory como todos los valores guardados corresponden a los mismos que se guardaron en la simulación de Mars.

Conclusiones

Como conclusión podemos decir que este proyecto es bastante largo, debido a que se debe hacer una implementación casi completa de un procesador monociclo, teniendo en cuenta muchos factores que afectan su ejecución. Para poder lograr esto se debe tener un buen conocimiento acerca del funcionamiento de un procesador, desde que se lee la instrucción hasta que se entregan los resultados. Algo también muy importante a tener en cuenta es que a la hora de hacer el programa en MIPS32, se manejan direcciones tanto de memoria como de instrucción distintas en Logisim y en Mars, por lo que a la hora de importar las instrucciones y los datos hay que tener mucho cuidado con esto.